

Durchführung der Studie

Technischer Aufbau

Hardware Setup

An Hardware werden ein Computer, ein Bildschirm und der uns zur Verfügung gestellte Tobii Pro Eyetracker benötigt. Dieser wird einfach am unteren Bildschirmrand festgeklebt. Nun müssen lediglich der Eyetracker und Bildschirm per Kabel mit dem Computer verbunden werden.

Weitergehend sollte man dieses Setup in einem ruhig gelegenen Raum mit Fenstern, die man abdunkeln kann, aufbauen, um äußere Faktoren wie Lärm, der die Probanden während des Versuches ablenken kann und stark wechselnde Lichtverhältnisse, welche die Funktion des Eyetrackers beeinträchtigen können, zu vermeiden.

Software Setup

Zunächst muss der [Tobi Pro Eye Tracker Manager](#) heruntergeladen werden und innerhalb des Managers daraufhin der Treiber für 'Tobii Pro Spark'. Nun kann der Eyetracker angeschlossen und mit dem Manager kalibriert werden (einmalig am Anfang zur Monitorkalibrierung; Kalibrierung sollte sonst für jeden Nutzer erneut durchgeführt werden).

In einem beliebigen Code Editor kann dann die tobii_research Python-Library mit 'pip install tobii_research' installiert werden (getestet haben wir es für Python Version 3.10 und 3.11.3).

Skript für Studiendurchführung

Für das folgende Skript muss ein txt-Dokument im selben Arbeitsverzeichnis mit dem Namen "ProbandID.txt" angelegt werden, in welchem die momentane Nummer des Probanden steht (initial also idealerweise 0 bzw. 1). Dieses Dokument stellt das kontinuierliche Inkrementieren der IDs sicher, auch über mehrere Sessions hinweg. Außerdem sollte das Programm mit "Escape" möglichst nur beendet werden, sobald ein neuer Proband kalibriert werden soll, sprich nach einem vollständigen Durchlauf eines Probanden, da sonst unvollständige Ergebnisse aufkommen (dies ist aber manuell zu beheben: einfach das txt-Dokument manuell um 1 verringern und die unvollständigen Testergebnisse löschen). Außerdem sind zwei Ordner mit Bildern nötig, deren Dateipfade den Variablen "folder_path" und "control_path" übergeben werden müssen. Die Bilder von "folder_path" sind die zur Studie verwendeten Bilder und die von "control_path" sind am Ende verwendete, nicht in der Studie verwendete Bilder, um zu testen ob der Proband auch aufmerksam zugeschaut hat. Somit sollte kein Bild in sowohl "control_path" als auch in "folder_path" auftauchen.

- Skript siehe Anhang

Datensicherung

Die Daten werden in CSV-Dateien abgespeichert - eine Datei pro Kombination aus Proband und Bild. Diese enthält ein Dictionary mit folgenden Keys:

Dictionary

- device_time_stamp:

Zeitsptempel des Eye-Trackers in Mikrosekunden

- system_time_stamp:

Zeitstempel des Systems in Mikrosekunden

Die folgenden Daten sind je für das **rechte** und **linke Auge** verfügbar. Hierzu muss lediglich '**left**' mit '**right**' ersetzt werden:

- left_gaze_point_on_display_area:
 - *(x, y) Koordinaten der Blickposition auf dem Display (normalisiert von 0 bis 1).*
 - *(0, 0) stellt die linke, obere Ecke des Bildschirms dar, (0.5, 0.5) die Mitte und (1, 1) die rechte, untere Ecke.*
 - *Zum umrechnen muss der normalisierte Wert mit der Pixelanzahl des Monitors multipliziert werden*
- left_gaze_point_in_user_coordinate_system:
 - *(x, y, z) Position des Blickpunkts (was schaut der Nutzer an) im Benutzersystem*
 - *Das Benutzersystem ermittelt die Position der Augen relativ zum Benutzer. Bezugspunkt ist ein Referenzpunkt im Gesicht.*
 - *Angaben in mm*
- left_gaze_point_validity:
 - *Gibt an, ob die Daten für das linke Auge gültig sind (0 = ungültig, 1 = gültig)*
 - *Kann z.B. auf blinzeln oder Aufnahmefehler hinweisen*
- left_pupil_diameter:
 - *Pupillendurchmesser (in mm)*
- left_pupil_validity:
 - *Gibt an, ob die Daten für die linke Pupille gültig sind (0 = ungültig, 1 = gültig)*
 - *Kann z.B. auf blinzeln oder Aufnahmefehler hinweisen*
- left_gaze_origin_in_user_coordinate_system:
 - *(x, y, z) Position des Blickstrahlursprungs im Benutzersystem*
 - *Das Benutzersystem ermittelt die Position der Augen relativ zum Benutzer. Bezugspunkt ist ein Referenzpunkt im Gesicht.*
 - *Angaben in mm*
- left_gaze_origin_validity:
 - *Gibt an, ob eine gültige Position für den Ursprung des Blickstrahls eines Auges bestimmt werden konnte (0 = ungültig, 1 = gültig)*

Weiterführendes

Bei Interesse finden sich mehr Informationen zur Funktionsweise von Eyetrackern oder ein Ansatz, um die Augenbewegungen bei Videos zu tracken, in unserem [HedgeDoc](#).

Durchführung

Studiendesign

Die Eye-Tracking Studie war in zwei Stimuli-Präsentationsphasen, welche durch eine optionale Pause voneinander getrennt waren, gegliedert. Nach der Einweisung der Versuchsperson (VP) und der Kalibrierung des Eye-Trackers wurden der VP 76 Stimuli aus einem Pool von 153 präsentiert. Diese wurden für mindestens 5 und maximal 15 Sekunden auf dem Monitor angezeigt. Falls die VP vor Ablauf der 15 Sekunden bereits das nächste Stimulus angezeigt bekommen wollte, so konnte sie dies mit dem Drücken der Leertaste tun. Nachdem die ersten 76 Stimuli in einer zufälligen Reihenfolge (um Müdigkeitsbias auf alle Stimuli zu verteilen) durchgelaufen waren, gab es die Möglichkeit einer kurzen Pause für die VP. Anschließend wurden die verbleibenden Stimuli in gleicher Art und Weise präsentiert. Nach Ablauf der zweiten Präsentationsphase wurde eine kurze Validitätskontrolle durchgeführt, um die Aufmerksamkeit der VP zu überprüfen.

Für die Studie hatten wir einen konkreten Zeitraum vom 17. bis zum 27. Juni festgelegt. Teilnehmende konnten sich dann für einen für sie passenden Zeit-Slot entscheiden. Stattgefunden hat dies im von uns eingerichteten Eye-Tracking Lab im zweiten Obergeschoss des Leo 3, in dem wir jedoch nur einen Eye-Tracker zur Verfügung hatten, weshalb Zeit-Slots immer nur einmalig vergeben werden konnten.

Teilnehmende

Nach einer ersten Recherche ergaben sich als Einschlusskriterien für die Teilnehmenden ein Alter von 18 bis 64 Jahren sowie das Fehlen schwerwiegender Seheinschränkungen.

Zur Bekanntmachung der Studie wurden unter anderem Plakate gedruckt und an verschiedenen stark frequentierten Orten der Universität und der Fachhochschule Münster aufgehängt, besonders viele am Leonardo-Campus und am Institut für Psychologie. Außerdem wurde eine ansprechend formulierte Textnachricht über WhatsApp in verschiedenen großen Gruppen verbreitet (Studienganggruppen, Wohnheimgruppen usw.). Diese Maßnahme erwies sich schlussendlich als die effektivste. Eine große Reichweite konnte ebenfalls durch die Veröffentlichung der Studie im wöchentlichen ASTA-Newsletter erzielt werden. Darüber hinaus wurde die Studie im Studienaushang des Fachbereichs Psychologie gelistet.

Da die Teilnehmenden fast ausschließlich aus Studierenden der Universität Münster rekrutiert wurden, beschränkt sich das tatsächliche Alter auf 19–29 Jahre. Unter den Teilnehmenden gaben etwa 60 % ihr Geschlecht als weiblich, 40 % als männlich und 0 % als divers/andere an. Alle Teilnehmenden waren deutschsprachig.

Ursprünglich meldeten sich 72 Personen für die Studie an, von denen schlussendlich 56 einen Termin buchten. 53 von ihnen erschienen zu dem gebuchten Termin und flossen somit in den Datensatz mit ein.

Als Anreiz konnten die Teilnehmenden zwischen einer Vergütung von 5 Euro in bar oder der Gutschrift von 0,5 Versuchspersonenstunden wählen. Letztere werden von Psychologie-

Studierenden benötigt, um den Bachelorabschluss zu erlangen. Die VP-Stunden wurden unter den Anmeldungen sechsmal in Anspruch genommen.

Stimuli und Materialien

Zur Auswahl der Stimuli einigten wir uns auf drei Kompositions- und vier Inhaltskategorien. Die Kompositionskategorien teilten sich in die Gruppe "Text", "kein Text" und "Bild mit Text" auf. Mit der Erfassung dieser Gruppen konnten wir das Blickverhalten von Social-Media-typischen multimodalen Inhalten untersuchen. Inhaltlich entschieden wir uns für die Bereiche "Politik", "Person", "Ort" und "Meme", wobei sich diese Kategorien überschneiden konnten. Während die Kategorien "Person" und "Ort" jeweils neutrale Situationen abgebildet haben, wurde die Auswahl der Memes und politischen Inhalte bewusst teils humorvoll, teils provokant getroffen, um unterschiedliche Reaktionen feststellen zu können. Um die Komplexität des KI-Trainings zu minimieren, entschlossen wir uns für ein einheitliches Bildformat von 800x800 Pixeln. Vorteile dieser quadratischen Form stellten besonders die hohe Verfügbarkeit von Material und ein guter Kompromiss zwischen Lesbarkeit und Dateigröße dar. Die Menge der Bilder wurde auf Grundlage des für die Studie vorgesehenen zeitlichen Rahmens getroffen. Da wir nach der Pilotstudie eine Länge von einer halben Stunde zur Reduzierung von Ermüdungserscheinungen für geeignet hielten, wählten wir eine Menge von 153 Bildern.

Die konkreten Bilder wurden unter Rücksicht auf Urheberrecht gewählt. 109 Bilder wurden aus den Creative-Commons ausgesucht. Dafür nutzten wir unter anderem die Googles Erweiterte Suche und Flickr Creative Commons. Die restlichen Bilder wurden ohne Lizenz ausgewählt, jedoch mit Link auf das Originalbild und Informationen zur Bounding-Box gespeichert. Dadurch lässt sich dieser Datensatz "dehydriert" veröffentlichen, also ohne die eigentlichen Bilder zu teilen, aber mit einer Möglichkeit, die Bilder automatisch aus ihrer ursprünglichen Quelle zu beziehen. Da diese Methode den Nachteil hat, dass der Rechteinhaber die Ursprungsbilder löschen könnte und sich die Bilder nicht rekonstruieren lassen, wurden vorwiegend Creative-Commons-Bilder ausgewählt.

Um die Kategorien und rechtlichen Rahmenbedingungen der einzelnen Bilder nachvollziehen zu können, wurde jedes Bild folgendermaßen benannt:

<ID>_<Tag1>_<Tag2>_<...>_.jpg

Dabei wurden die Tags "text", "textimg", "meme", "ort", "person", "politik" und "ncc"(non-creative-commons) genutzt.

Versuchsablauf

Nachdem eine teilnehmende Versuchsperson durch den Anmeldeprozess gegangen ist und zu seinem ausgewählten Slot erschienen ist, gab es eine kurze Einweisung in den Ablauf der Studie und die beabsichtigte Nutzung der Daten. Es wurde darauf geachtet, dass alle für den Versuchsablauf relevanten Information kommuniziert werden. Darunter zählen die korrekte Kalibrierung des Eye-Trackers, das Ruhighalten des Kopfes nach Kalibrierung, die optionale Pause nach der Hälfte der Zeit und die Möglichkeit des Weiterklickens der Stimuli (was einer Simulation eines Social Media-Feed nahekommmt).

Im Anschluss bekam die Versuchsperson nochmal eine schriftliche Teilnehmerinformation und Datenschutzerklärung, die gelesen und unterschrieben werden musste.

Mit dem Unterschreiben dieser Dokumente und der Einweisung konnte die Präsentationsphase der Stimuli starten. Die Versuchspersonen haben keine konkrete Aufgabe (z.B. das Zählen

bestimmter Elemente) bekommen, da in diesem Falle die Gefahr bestünde, unnatürliches Blickverhalten zu provozieren. Um möglichst natürliches Blickverhalten (wie beim Scrollen durch einen Social Media Feed) und einfaches “Durchklicken” der Stimuli zu vermeiden, haben wir eine kurze Validitätskontrolle am Ende der Präsentationsphase eingebaut, die zu Beginn auch kommuniziert wurde. Und zwar wurden den Teilnehmenden 5 Stimuli aus dem Pool der bereits präsentierten Stimuli und 5 gänzlich neue Stimuli gezeigt. Diese mussten nun korrekt in “bereits gesehen” und “neu” klassifiziert werden, um einen validen Datensatz zu erhalten. Die Pause, die sich Teilnehmende nach der Hälfte der Zeit nehmen konnten, war optional und die Präsentationsphase konnte somit nahtlos fortgesetzt werden, sofern dies gewünscht war.

Nach Durchlauf des letzten Stimuli war die Teilnahme der Versuchsperson praktisch beendet. Die VP konnte sich zu guter Letzt noch für die Art der Vergütung entscheiden (0,5 VP Stunden oder 5 €).

Organisation

Die Teilnehmenden konnten sich zunächst über ein „Microsoft Forms“-Formular anmelden. Dieses wurde als Link in den Textnachrichten sowie als QR-Code auf den Plakaten bereitgestellt. Hier konnten sie ihre grundlegenden Daten angeben (Name, Geburtsdatum, Geschlecht, E-Mail-Adresse, Angaben zu Leseschwierigkeiten, Teilnahmevergütung, Datenschutzerklärung).

Anschließend erhielten sie automatisch eine personalisierte E-Mail mit einem Calendly-Link, in dem ihre Daten bereits vorausgefüllt waren. So mussten die Teilnehmenden nur noch einen Termin im von uns vorgegebenen Zeitraum auswählen. Dieser zweischrittige Prozess hatte einerseits den Vorteil, dass sich die technischen Limitationen eines einfachen Formulars zur Terminkoordination umgehen ließen. Andererseits wurden durch die geringe zusätzliche Schwelle besonders unmotivierte Teilnehmende frühzeitig gefiltert, was die No-Show-Rate reduzierte.

Termine wurden unter der Woche täglich (mit Ausnahme eines Feiertags) von 9 bis 18 Uhr vergeben. Als Zeitpuffer gab es täglich drei Slots, die nicht buchbar waren.

Für unsere interne Zeitplanung wurde ein Notion-Board eingerichtet. Dabei teilten wir jeden Tag in zwei Schichten ein, in denen jeweils zwei Personen anwesend waren.

Im Voraus wurden die zu unterschreibende Teilnahmeinformation, Vorlagen zur Bescheinigung der VP-Stunden sowie eine Unterschriftenliste, die den Erhalt der Teilnahmevergütung quittierte, ausgedruckt. Zudem wurden Wegweiser angefertigt und im Gebäude ausgehängt. Die 5-Euro-Scheine lagen in einer Schublade bereit.

Datenerhebung

- Datenerfassungsprotokoll

Die Eyetracking-Daten wurden als Scanpaths in csv-Dateien gespeichert. Jede Scanpath-Datei besitzt folgende Header:

device_time_stamp,system_time_stamp,left_gaze_point_on_display_area,left_gaze_point_in_user_coordinate_system,left_gaze_point_validity,left_pupil_diameter,left_pupil_validity,left_gaze_origin_in_user_coordinate_system,left_gaze_origin_validity,right_gaze_point_on_display_area,right_gaze_point_in_user_coordinate_system,right_gaze_point_validity,right_pupil_diameter,right_pupil_validity,right_gaze_origin_in_user_coordinate_system,right_gaze_origin_validity

Die Dateien wurden zunächst jeweils im Format Proband<ProbandID>_<BildName>.csv gespeichert und später zur erleichterten Sortierung und Verarbeitung in

P<ProbandID>*<*BildID*>*<BildTagBinaryCode>.csv umbenannt. Der Binary-Code ist eine Repräsentation der Tags, die das Bild enthält, wobei ein Bit jeweils anzeigt, ob das Bild zum wesentlichen Anteil Text, Ort, Meme, Politik oder Person enthält.

Die Dateien wurden über Git getrackt und das korrespondierende Github-Repository täglich geupdatet. Außerdem erstellten wir regelmäßig lokale Sicherheitskopien und überprüften, ob die Daten plausibel erschienen.

Qualitätssicherung

Um einen möglichst reibungslosen Ablauf der Studie sicherzustellen, führten wir wenige Wochen vor geplantem Studienbeginn eine Pilotstudie an uns selbst durch. Dabei nutzten wir die gleichen Stimuli, wie sie in der Studie verwendet werden sollten unter gleichen Bedingungen und simulierten nacheinander einen kompletten Durchlauf. Daraufhin nahmen wir eine Reihe von Anpassungen vor, unter anderem Markierungen auf dem Boden, die die korrekte Position des Stuhls zeigen sollten, eine Pause beim Erreichen der Hälfte der Bilder und Änderungen in der Bedienung.

Die Teilnehmenden erhielten bei der Studie einheitliche Instruktionen, die die Qualität der Scanpaths gewährleisten sollten. Darunter fielen die korrekte Positionierung des Stuhles, die Augen stets auf dem Bildschirm zu halten und eine Hand auf der Space-Taste zu behalten, damit zum nächsten Bild geskippt werden konnte, ohne vom Bildschirm fortzublicken.

Wegen eines Fehlers in der Bedienung verloren wir den ersten Datensatz, der automatisch überschrieben wurde. Allerdings konnte noch am gleichen Tag eine Ausfallsicherung implementiert werden, die durch Zeitstempel im Dateinamen erfolgreich weiteren Datenverlust durch Überschreibung verhindern konnte. Einige Male wurde der Eyetracker fehlerhaft gestartet, weswegen irrelevante Dateien erzeugt wurden. Diese Dateien wurden anschließend aussortiert und der Index der darauffolgenden Dateien angepasst.

Ethik und Compliance

Um allen ethischen Anforderungen an die Studie gerecht zu werden, haben wir einen ausführlichen Ethikantrag an die Ethikkommission des Fachbereichs 4 gesendet. Diese hat unser Studienvorhaben als ethisch unbedenklich eingestuft, wodurch der Durchführung nichts mehr im Wege stand.

Die Daten wurden zunächst pseudonymisiert erhoben, um Versuchspersonen noch die Möglichkeit zu geben ihre Daten löschen zu lassen. Nach Ablauf der Studie wurden diese Daten anonymisiert, wodurch kein Rückschluss mehr auf die einzelnen Teilnehmenden mehr möglich ist.

Die Versuchspersonen hatten zu jeder Zeit die Möglichkeit, die Teilnahme zu verweigern oder den Versuch abubrechen ohne jegliche Nachteile zu erfahren.

Reflexion

Größtenteils war die Datenerhebung erfolgreich und der Ablauf der Studie ohne größere Probleme.

Abgesehen von dem Verlust eines Datensatzes verlief die Datenerhebung reibungslos. Die Scanpaths wurden akkurat vom Eyetracker erfasst und Tendenzen bei der Betrachtung konnten klar aus späteren Visualisierungen entnommen werden.

Schwierigkeiten gab es bei der Koordination der Anwesenheiten im Eyetracking-Lab. Hier musste einige Male kurzfristig eingesprungen werden, um nicht zugeteilte Schichten zu übernehmen. Trotzdem waren schließlich zu jedem Zeitfenster zwei von uns vor Ort.

An den insgesamt vierzehn angebotenen Tagen war die Anzahl gebuchter Termine außerdem stark ungleichmäßig. Teilweise waren die Tage voll ausgebucht, an einem Tag wurde aber fast kein Slot gebucht. Frühere Werbung hätte womöglich noch mehr Slots füllen können.

Anhang

Skript für Studiendurchführung

```
#Die verwendeten libraries
import tobii_research as tr
import pygame
import random
import os
import csv
import time

#Probanden anlegen/inkrementieren, wie oben beschrieben
with open('ProbandID.txt', 'r', encoding='utf-8') as f:
    proband = f.read().strip()
    proband = int(proband)

#Ordner mit Bildern festlegen
list = []
folder_path = 'current_images/'
control_path = 'control_images/'
max_time = 15 # Max Anzeigedauer in s
min_time = 5  # Min Anzeigedauer in s

#Aufnahme starten und aufgenommene Daten erhalten
def gaze_data_callback(gaze_data):
    #if(index < len(images)):
    #    gaze_data["Image"] = images[index]
    #else: gaze_data["Image"] = None
    list.append(gaze_data)

#Mit Eye-Tracker verbinden
def find_tracker():

    found_eyetrackers = tr.find_all_eyetrackers()
```

```

if (len(found_eyetrackers) > 0):
    pTracker = found_eyetrackers[0]
    print("Eye-Tracker connected: " + pTracker.device_name)
    return pTracker
else:
    print("No Eye-Trackers found!")
    return None

#Eye-Tracker per Code kalibrieren
def calibrate_tracker(pTracker):
    screen_width, screen_height = screen.get_size()

    calibration = tr.ScreenBasedCalibration(pTracker)
    calibration.enter_calibration_mode()

    points_to_calibrate = [(0.5, 0.5), (0.1, 0.1), (0.1, 0.9), (0.9, 0.1), (0.9, 0.9)]
    for point in points_to_calibrate:
        x = int(point[0] * screen_width)
        y = int(point[1] * screen_height)
        r = 10

        screen.fill((0, 0, 0))
        pygame.draw.circle(screen, 'red', (x - r, y - r), r)
        pygame.display.flip()

        time.sleep(3)

        if calibration.collect_data(point[0], point[1]) != tr.CALIBRATION_STATUS_SUCCESS:
            calibration.collect_data(point[0], point[1])

    calibration_result = calibration.compute_and_apply()

    calibration.leave_calibration_mode()

#Bilder einheitlich skalieren
def scale_image(pImage):
    width, height = pImage.get_size()
    swidth, sheight = screen.get_size()

    if swidth - width < sheight - height:
        pImage = pygame.transform.smoothscale(pImage, (swidth, int((swidth / width) * height)))
    else:

```



```

        pImage = pygame.transform.smoothscale(pImage, (int((sheight / height) * width),
sheight))

    return pImage

def next_image():
    global index, images, image, list, proband, skip

    index += 1

    if index < len(images):
        if index == round(len(images) / 2):
            curtime = time.asctime().split(' ')[3].split(':')[0] + '_' + time.asctime().split('
')[3].split(':')[1] + '_' + time.asctime().split(' ')[3].split(':')[2]

            with open('Proband' + str(proband) + '_' + curtime + '_' + images[index -
1].split('.')[0] + '.csv', 'w', newline='', encoding='utf-8') as csvfile:
                # Bestimme die Feldnamen basierend auf den Keys des ersten Dictionaries
                feldnamen = list[0].keys()
                writer = csv.DictWriter(csvfile, fieldnames=feldnamen)
                # Schreibe die Kopfzeile
                writer.writeheader()
                # Schreibe die Datenzeilen
                writer.writerows(list)
            pause()
        else:
            image = pygame.image.load(r'' + folder_path + images[index])
            image = scale_image(image)

            curtime = time.asctime().split(' ')[3].split(':')[0] + '_' + time.asctime().split('
')[3].split(':')[1] + '_' + time.asctime().split(' ')[3].split(':')[2]

            with open('Proband' + str(proband) + '_' + curtime + '_' + images[index -
1].split('.')[0] + '.csv', 'w', newline='', encoding='utf-8') as csvfile:
                # Bestimme die Feldnamen basierend auf den Keys des ersten Dictionaries
                feldnamen = list[0].keys()
                writer = csv.DictWriter(csvfile, fieldnames=feldnamen)
                # Schreibe die Kopfzeile
                writer.writeheader()
                # Schreibe die Datenzeilen
                writer.writerows(list)

    list = []

    pygame.time.set_timer(skippable, min_time * 1000)

```

```

pygame.time.set_timer(force_skip, max_time * 1000)

elif index == len(images):
    curtime = time.asctime().split(' ')[3].split(':')[0] + '_' + time.asctime().split(' ')[3].split(':')[1] + '_' + time.asctime().split(' ')[3].split(':')[2]

    with open('Proband' + str(proband) + '_' + curtime + '_' + images[index - 1].split('.')[0] + '.csv', 'w', newline='', encoding='utf-8') as csvfile:
        # Bestimme die Feldnamen basierend auf den Keys des ersten Dictionaries
        felddaten = list[0].keys()
        writer = csv.DictWriter(csvfile, fieldnames=felddaten)
        # Schreibe die Kopfzeile
        writer.writeheader()
        # Schreibe die Datenzeilen
        writer.writerows(list)

    control()

    image = pygame.font.Font(None, 74).render("Neuen Probanden kalibrieren? Dann jetzt '->' drücken", True, (255, 255, 255))

    pygame.time.set_timer(skippable, min_time*1000)
    pygame.time.set_timer(force_skip, 0)

else:
    calibrate_tracker(tracker)
    index = 0
    random.shuffle(images)
    image = pygame.image.load(r'' + folder_path + images[index])
    image = scale_image(image)
    list = []
    proband += 1
    pygame.time.set_timer(skippable, min_time * 1000)
    pygame.time.set_timer(force_skip, max_time * 1000)

skip = False

#Pause in der Mitte der Studie
def pause():
    global image, paused, tracker

    tracker.unsubscribe_from(tr.EYETRACKER_GAZE_DATA, gaze_data_callback)

```

```

pygame.time.set_timer(skippable, 0)
pygame.time.set_timer(force_skip, 0)

paused = True
image = pygame.font.Font(None, 74).render("Pausiert. Drücke 'R' zum fortfahren", True, (255, 255, 255))

#Pause beenden
def resume():
    global image, paused, list, folder_path, images, index, tracker

    tracker.subscribe_to(tr.EYETRACKER_GAZE_DATA, gaze_data_callback, as_dictionary=True)

    calibrate_tracker(tracker)

    image = pygame.image.load(r'' + folder_path + images[index])
    image = scale_image(image)

    pygame.time.set_timer(skippable, min_time * 1000)
    pygame.time.set_timer(force_skip, max_time * 1000)

    list = []
    paused = False

#Validitätskontrolle am Ende der Studie
def control():
    global control_img, images, proband

    tracker.unsubscribe_from(tr.EYETRACKER_GAZE_DATA, gaze_data_callback)

    if (len(control_img) <= len(images)):
        real = random.sample(range(len(images)), k=len(control_img))
        test = control_img.copy()
        for ind in real:
            test.append(images[ind])
        random.shuffle(test)

        first = pygame.font.Font(None, 74).render("Wurden die nachfolgenden Bilder in der Studie bereits gezeigt?", True, (255, 255, 255))
        second = pygame.font.Font(None, 74).render("'J' für Ja, 'N' für Nein, 'S' zum Starten", True, (255, 255, 255))

        screen.fill((0, 0, 0))

```

```

        screen.blit(first, (screen.get_width() / 2 - first.get_width() / 2, screen.get_height()
/ 2 - first.get_height() / 2 - 50))

        screen.blit(second, (screen.get_width() / 2 - second.get_width() / 2,
screen.get_height() / 2 - second.get_height() / 2 + 50))

    pygame.display.flip()

    waiting = True
    while waiting:
        for event in pygame.event.get():
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_s:
                    waiting = False

        pygame.display.update()

    correctness = []
    tmp = 0
    if test[tmp] in images:
        img = pygame.image.load(r'' + folder_path + test[tmp])
    else:
        img = pygame.image.load(r'' + control_path + test[tmp])
    img = scale_image(img)
    while tmp < len(test):
        screen.fill((0, 0, 0))

        screen.blit(img, (screen.get_width() / 2 - img.get_width() / 2, screen.get_height()
/ 2 - img.get_height() / 2))

        for event in pygame.event.get():
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_j:
                    if test[tmp] in images:
                        correctness.append({'Bild':test[tmp], 'Erwartet':True,
'Angegeben':True})
                    else:
                        correctness.append({'Bild':test[tmp], 'Erwartet':False,
'Angegeben':True})

                tmp = tmp + 1
            if tmp < len(test):
                if test[tmp] in images:
                    img = pygame.image.load(r'' + folder_path + test[tmp])
                else:
                    img = pygame.image.load(r'' + control_path + test[tmp])
                img = scale_image(img)

```

```

        if event.key == pygame.K_n:
            if test[tmp] in images:
                correctness.append({'Bild':test[tmp], 'Erwartet':True,
'Angegeben':False})
            else:
                correctness.append({'Bild':test[tmp], 'Erwartet':False,
'Angegeben':False})

            tmp = tmp + 1
            if tmp < len(test):
                if test[tmp] in images:
                    img = pygame.image.load(r'' + folder_path + test[tmp])
                else:
                    img = pygame.image.load(r'' + control_path + test[tmp])
                img = scale_image(img)

pygame.display.update()

curtime = time.asctime().split(' ')[3].split(':')[0] + '_' + time.asctime().split('
')[3].split(':')[1] + '_' + time.asctime().split(' ')[3].split(':')[2]

with open('Proband' + str(proband) + '_' + curtime + '_Valididitätskontrolle.csv', 'w',
newline='', encoding='utf-8') as csvfile:

    # Bestimme die Feldnamen basierend auf den Keys des ersten Dictionaries
    feldnamen = correctness[0].keys()
    writer = csv.DictWriter(csvfile, fieldnames=feldnamen)

    # Schreibe die Kopfzeile
    writer.writeheader()

    # Schreibe die Datenzeilen
    writer.writerows(correctness)

tracker.subscribe_to(tr.EYETRACKER_GAZE_DATA, gaze_data_callback,as_dictionary=True)

#Bilder initialisieren
images = [f for f in sorted(os.listdir(folder_path)) if f.lower().endswith('.png') or
f.lower().endswith("jpeg") or f.lower().endswith("jpg")]
if not images:
    print("Keine Bilder im Ordner gefunden.")

control_img = [f for f in sorted(os.listdir(control_path)) if f.lower().endswith('.png') or
f.lower().endswith("jpeg") or f.lower().endswith("jpg")]
if not control_img:

```

```

print("Keine Bilder im Kontrollordner")

random.shuffle(images)
index = 0

#PyGame initialisieren, bevor der tracker kalibriert werden kann
pygame.init()
screen = pygame.display.set_mode((0,0), pygame.FULLSCREEN)
tracker = None
tracker = find_tracker()
calibrate_tracker(tracker)

force_skip = pygame.USEREVENT
skippable = pygame.USEREVENT + 1

image = pygame.image.load(r'' + folder_path + images[index])
image = scale_image(image)

tracker.subscribe_to(tr.EYETRACKER_GAZE_DATA, gaze_data_callback,as_dictionary=True)
skip = False
paused = False
run = True
pygame.time.set_timer(skippable, min_time * 1000)
pygame.time.set_timer(force_skip, max_time * 1000)
while run:
    screen.fill((0, 0, 0))
    screen.blit(image, (screen.get_width() / 2 - image.get_width() / 2, screen.get_height() / 2
- image.get_height() / 2))

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            run = False

        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                run = False

            if (event.key == pygame.K_RIGHT or event.key == pygame.K_SPACE) and skip and not
paused:
                next_image()

            if event.key == pygame.K_p and not paused:
                pause()

```

```
        if event.key == pygame.K_r and paused:
            resume()

        elif event.type == force_skip:
            next_image()

        elif event.type == skippable:
            skip = True
            pygame.time.set_timer(skippable, 0)

    pygame.display.update()

    tracker.unsubscribe_from(tr.EYETRACKER_GAZE_DATA, gaze_data_callback)
    pygame.quit()

    with open ('ProbandID.txt', 'w', encoding='utf-8') as f:
        f.write(str(proband + 1))
```

[Pilottesting.py](#)